

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«Сибирский государственный университет науки и технологий
имени академика М.Ф. Решетнева»**

Институт информатики и телекоммуникаций

Кафедра информатики и вычислительной техники

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

Теория информации

Линейные групповые коды

Руководитель

подпись, дата

А.Н. Бочаров

инициалы, фамилия

Обучающийся БПИ24-02, 24151430

номер группы, зачетной книжки

подпись, дата

В.Т. Гордов

инициалы, фамилия

Красноярск 2026 г.

ЦЕЛЬ РАБОТЫ

Закрепление знаний по методам кодирования информации.

ЗАДАНИЕ

1. Построить линейный групповой код, способный исправлять одиночную ошибку. Вариант взять аналогичный из лабораторной работы №4.
2. Привести пример построения 10 кодовых комбинаций.
3. Показать процедуру исправления ошибки в заданном разряде.
4. Составить программу, кодирующую и декодирующую кодовую комбинацию с целью обнаружения и исправления одиночной ошибки.

ХОД РАБОТЫ

Для выполнения лабораторной работы было создано консольное приложение, написанное на языке Python. Исходный код программы представлен ниже.

Файл **header.py**

```
import math
import random

class LinearGroupCoder:
    def __init__(self, num_messages):
        """Инициализация параметров кода ЛГК. 256 сообщений для 5 варианта."""
        # кол-во информационных разрядов
        self.n_i = int(math.log2(num_messages))
        # кол-во контрольных разрядов
        self.n_k = math.ceil(math.log2((self.n_i + 1) + math.log2(self.n_i + 1)))
        # Проверочная матрица П
        self.P = [
            [0, 0, 1, 1], # 1
            [0, 1, 0, 1], # 2
            [0, 1, 1, 0], # 3
            [1, 0, 0, 1], # 4
            [1, 0, 1, 0], # 5
            [1, 1, 0, 0], # 6
            [0, 1, 1, 1], # 7
            [1, 0, 1, 1] # 8
        ]

    def generate_data_string(self):
        """Генерация случайной последовательности данных."""
        return "".join(random.choice("01") for _ in range(self.n_i))

    def encode(self, data_string):
        """Кодирование ЛГК."""
        u = [int(bit) for bit in data_string]
        # Информационная часть (единичная матрица I просто переносит биты)
        code = u.copy()
        # Проверочная часть (умножение вектора U на матрицу П по модулю 2)
        for j in range(self.n_k):
            # Суммируем (XOR) элементы столбца j матрицы P, если соответствующий бит u[i] ==
            1
            check_bit = sum(u[i] * self.P[i][j] for i in range(self.n_i)) % 2
            code.append(check_bit)

        return "".join(map(str, code))

    def decode(self, code_str):
        """Декодирование ЛГК и исправление одиночной ошибки."""
```

```

# Вектор кода
v = [int(bit) for bit in code_str]
# Извлекаем принятые информационные и контрольные биты
u_recv = v[:self.n_i]
c_recv = v[self.n_i:]
# Вычисляем синдрома (XOR сумма всех битов)
syndrome = []
for j in range(self.n_k):
    s_bit = (sum(u_recv[i] * self.P[i][j] for i in range(self.n_i)) + c_recv[j]) % 2
    syndrome.append(s_bit)
status = ""
corrected_code = v.copy()
# Анализ синдрома
if all(bit == 0 for bit in syndrome):
    status = "Ошибок нет."
else:
    # Ищем, какому столбцу проверочной матрицы H соответствует синдром.
    # Проверяем информационную часть (совпадает ли синдром со строкой матрицы P)
    error_pos = -1
    for i in range(self.n_i):
        if self.P[i] == syndrome:
            error_pos = i
            break
    # Проверяем контрольную часть (совпадает ли синдром с единичной матрицей)
    if error_pos == -1:
        for j in range(self.n_k):
            # Генерируем столбец единичной матрицы
            i_col = [1 if k == j else 0 for k in range(self.n_k)]
            if i_col == syndrome:
                error_pos = self.n_i + j
                break
    if error_pos != -1:
        status = f"Одиночная ошибка в позиции {error_pos + 1}. Исправление выполнено."
        corrected_code[error_pos] ^= 1 # Инвертируем ошибочный бит
    else:
        status = f"Неизвестный синдром {syndrome}. Множественная ошибка!"
# Извлечение информационной части из исправленного кода
data = "".join(map(str, corrected_code[:self.n_i]))
final_code_str = "".join(map(str, corrected_code))
return status, final_code_str, data

```

Файл **main.py**

```
from header import LinearGroupCoder
```

```

def simulate_error(code_str, position):
    """Внесение ошибки в строку."""
    code_list = list(code_str)
    idx = position - 1
    if 0 <= idx < len(code_list):

```

```

    code_list[idx] = '1' if code_list[idx] == '0' else '0'
return "".join(code_list)

def main():
    # 5 вариант: 256 сообщений
    coder = LinearGroupCoder(num_messages=256)

    while True:
        print("\n--- ЛАБОРАТОРНАЯ РАБОТА № 5 ---")
        print("1. Расчёт параметров и вывод порождающей матрицы G")
        print("2. Пример построения 10 кодовых сообщений")
        print("3. Демонстрация исправления одиночной ошибки")
        print("0. Выход")

        choice = input("Выбор: ")

        if choice == "1":
            print("\n" + "="*40)
            print("ПАРАМЕТРЫ ЛГК")
            print("="*40)
            print(f"Количество сообщений (N): {256}")
            print(f"Информационных разрядов (n_и): {coder.n_i}")
            print(f"Контрольных разрядов (n_к): {coder.n_k}")
            print(f"Порождающая матрица G = [ I | P ]:")
            # Выводим матрицу G для наглядности (I - единичная 8x8, P - 8x4)
            for i in range(coder.n_i):
                i_row = ["1" if k == i else "0" for k in range(coder.n_i)]
                p_row = [str(x) for x in coder.P[i]]
                print(f"{' '.join(i_row)} | {' '.join(p_row)}")

        elif choice == "2":
            print("\n" + "="*40)
            print("ПОСТРОЕНИЕ 10 СЛУЧАЙНЫХ СООБЩЕНИЙ")
            print("="*40)
            print(f"{'№':<3} | {'Инфо-данные':<12} | {'Систематический ЛГК (Инфо + Проверка)'}")
            print("-" * 65)
            for i in range(1, 11):
                data = coder.generate_data_string()
                encoded = coder.encode(data)
                formatted_encoded = f"{encoded[:coder.n_i]} {encoded[coder.n_i:]}"
                print(f"{i:<3} | {data:<12} | {formatted_encoded}")

        elif choice == "3":
            print("\n" + "="*40)
            print("СИМУЛЯТОР КАНАЛА С ПОМЕХАМИ")
            print("="*40)
            data = coder.generate_data_string()
            encoded = coder.encode(data)
            print(f"Сгенерированные данные: {data}")
            print(f"Отправленный в канал ЛГК: {encoded}")

```

```

err_input = input("\nУкажите позицию для внесения ошибки (от 1 до 12): ").strip()
err_pos = int(err_input)
received = simulate_error(encoded, err_pos)
print(f"\nПолученный из канала код (с ошибкой): {received}")

status, corrected, decoded_data = coder.decode(received)
print(f"\nСтатус декодирования: {status}")
print(f"Исправленный код: {corrected}")
print(f"Итоговая последовательность данных: {decoded_data}")

elif choice == "0":
    break
else:
    print("Неверный ввод.")

if __name__ == "__main__":
    main()

```

В коде представленной программы был реализован класс **LinearGroupCoder**, предназначенный для кодирования и декодирования информации методом Хемминга.

Взаимодействие с функциональными методами происходит с помощью консольного меню (Рисунок 1).

```

--- ЛАБОРАТОРНАЯ РАБОТА № 5 ---
1. Расчёт параметров и вывод порождающей матрицы G
2. Пример построения 10 кодовых сообщений
3. Демонстрация исправления одиночной ошибки
0. Выход
Выбор: 

```

Рисунок 1 — Меню программы

Задание 1

Аналогично 5 варианту лабораторной работы №4, код программы предусматривает посылку 256 сообщений. Исходя из этого условия, при инициализации программы следующие параметры линейного группового кода:

- Количество информационных разрядов кода $n_u = \log_2 N$;
- Количество контрольных разрядов $n_k = \log_2((n_i + 1) + \log_2(n_i + 1))$;
- Производящая матрица, состоящая из информационной части И и проверочной части П.

При выборе 1 пункта программы данные параметры выводятся в консоль (Рисунок 2).

Выбор: 1

=====

ПАРАМЕТРЫ ЛГК

=====

Количество сообщений (N): 256

Информационных разрядов (n_и): 8

Контрольных разрядов (n_к): 4

Порождающая матрица $G = [I \mid P]$:

1	0	0	0	0	0	0	0	0	0	0	0	1	1
0	1	0	0	0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	0	0	1	0	0
0	0	0	0	1	0	0	0	0	0	0	1	0	0
0	0	0	0	0	1	0	0	0	0	0	1	0	0
0	0	0	0	0	0	1	0	0	0	0	1	1	0
0	0	0	0	0	0	0	1	0	0	0	0	1	1
0	0	0	0	0	0	0	0	1	0	0	1	0	1

Рисунок 2 — Вывод параметров ЛГК

Задание 2

При выборе 2 пункта программа последовательно генерирует 10 случайных 8-ми разрядных комбинаций, после чего строит ЛГК для каждого из них. Результаты представлены на Рисунке 3.

=====

ПОСТРОЕНИЕ 10 СЛУЧАЙНЫХ СООБЩЕНИЙ

=====

№	Инфо-данные	ЛГК (Инфо + Проверка)
1	00101110	00101110 0111
2	00110110	00110110 0100
3	11100100	11100100 1100
4	00001100	00001100 0110
5	10011101	10011101 0111
6	10111001	10111001 1101
7	11111101	11111101 0100
8	11010111	11010111 1111
9	01110100	01110100 0110
10	01110010	01110010 1101

Рисунок 3 — Построение ЛГК для 10 сообщений

Задание 3

Для построения ЛГК предварительно составим проверочную матрицу Π размерами 8×4 . Строки матрицы соответствуют информационным разрядам a_n , а столбцы — контрольным p_m . При этом вес каждой строки матрицы Π должен быть не менее $W_{\Pi} \geq d_{min} - 1$. Матрица представлена в Таблице 1.

Таблица 1

0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
0	1	1	1
1	0	1	1

Исходя из столбцов матрицы Π формируются уравнения для вычисления контрольных разрядов путём сложения по модулю 2:

- $p_1 = a_4 + a_5 + a_6 + a_8$;
- $p_2 = a_2 + a_3 + a_6 + a_7$;
- $p_3 = a_1 + a_3 + a_5 + a_7 + a_8$;
- $p_4 = a_1 + a_2 + a_4 + a_7 + a_8$.

Исходная последовательность данных: **01000101**. Путём последовательного вычисления строятся контрольные биты, которые дописываются после информационной части для формирования ЛГК.

Сформированный код: **010001010010**

Симуляция одиночной ошибки:

Представим, что ошибка произошла в 3-м бите кода, заменив его значение с **0** на **1**. Код после прохождения через канал выглядит так: **011001010010**.

Для обнаружения ошибки строится контрольная матрица H путём транспонирования матрицы Π , дополненной единичной матрицей вида $n_k * n_k$. Матрица представлена в Таблице 2.

Таблица 2

1	1	0	1	0	0	1	1	1	0	0	0
1	0	1	0	1	0	1	1	0	1	0	0
0	1	1	0	0	1	1	0	0	0	1	0
1	0	0	1	1	1	0	1	0	0	0	1

Исправление ошибки происходит по следующему алгоритму:

1. ЛГК разбивается на информационные биты **01100101** и контрольные **0010**.
2. Биты синдрома вычисляются аналогично значениям контрольных битов с дополнительным прибавлением значений контрольных битов.
Так, $s_1 = p_1 + a_4 + a_5 + a_6 + a_8 = 0 + 0 + 0 + 1 + 1 = 0$
3. После вычисления всех битов полученный синдром выглядит так:
 $S = 0110$.
4. Декодер ищет вектор синдрома среди столбцов контрольной матрицы H . 3 столбец $[0110]$ совпадает, что указывает на позицию ошибки в коде — 3 бит.
5. Найденный ошибочный разряд инвертируется, после чего код принимает вид **010001010010**. Соответственно, информационная часть принимает корректное значение **01000101**.

Задание 4

При выборе 3 пункта программа генерирует последовательность данных и строит по ней ЛГК. Затем от пользователя требуется указать позицию кодовой строки, на которой должна быть симулирована ошибка.

После ввода данных пользователем символ на выбранной позиции инвертируется, симулируя возникновение ошибки при прохождении кода через канал связи. Далее предпринимается попытка обнаружить и исправить ошибку и декодировать кодовую последовательность в информационную строку. Демонстрация работы представлена на Рисунке 4.

```
=====
СИМУЛЯТОР КАНАЛА С ПОМЕХАМИ
=====
Сгенерированные данные: 11001001
Отправленный в канал ЛГК: 110010010111

Укажите позицию для внесения ошибки (от 1 до 12): 3

Полученный из канала код (с ошибкой): 111010010111

Статус декодирования: Одиночная ошибка в позиции 3. Исправление выполнено.
Исправленный код: 110010010111
Итоговая последовательность данных: 11001001
```

Рисунок 4 — Симуляция одиночной ошибки в ЛГК

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Что такое линейные групповые коды?

Это коды, в которых корректирующие биты есть линейные комбинации информационных битов.

2. Как строится производящая матрица G ?

Производящая матрица имеет размер $n_i * n$ и вид $G = [I|P]$.

Информационная часть I — единичная матрица $n_i * n_i$ в каноничной форме.

Проверочная часть P — вес каждой строки соответствует $W_P \geq d_{min} - 1$.

3. Как строится контрольная матрица H ?

$H = [P^T | I_{n_k}]$ — транспонируем матрицу P и дополняем её единичной матрицей $n_k * n_k$.

4. Как происходит декодирование линейного группового кода?

Путём вычисления синдрома. В случае, если синдром ненулевой, он сравнивается со столбцами матрицы H . Номер столбца, совпадающего с вектором синдрома, является позицией ошибки в ЛГК.

5. Что такое систематический код?

Код, в котором информационные и корректирующие биты занимают строго определённые позиции. Таковым является ЛГК.

6. В чём суть методики построения систематического кода?

1. В качестве первого вектора берется нулевой вектор.

2. Составляется производящая матрица G вида $n_i * n$.

В качестве строк берутся любые ненулевые линейно-независимые n значные векторы, отстоящие друг от друга не менее, чем на заданное кодовое расстояние.

3. Остальные кодовые вектора числом $2^{n_i} - n_i - 1$ получаются как линейные комбинации векторов, входящих в производящую матрицу.

4. Для обнаружения ошибок в кодовых комбинациях составляется множество проверочных векторов, ортогональных векторам кода. Из проверочных векторов составляют проверочную матрицу H вида $n_k * n_k$. Её строками являются любые линейно-независимые комбинации проверочных векторов.

7. Какие методы декодирования систематического кода вы знаете?

1. Метод полной кодовой таблицы.

2. Метод проверки на чётность.

3. Метод полного корректирующего вектора.

8. Как проводится декодирование систематического кода по методу полной кодовой таблицы?

Строится таблица, первая строка — все разрешённые кодовые комбинации начиная с нулевой. Первый столбец — все возможные векторы ошибок. Остальные ячейки заполняются суммированием по модулю 2 вектором ошибок со столбцом разрешенной комбинации.

Общий метод декодирования состоит в том, что приняв некий вектор V_x его отыскивают в полной кодовой таблице и соотносят с тем кодовым вектором, который стоит в верхней строке столбца.

Операция имеет вид $V_x + e_i = V_k$.

9. Как проводится декодирование систематического кода по методу проверки на чётность?

На основе свойства ортогональности проверочных векторов составляются правила проверки (уравнения). В эти уравнения подставляются биты принятого сообщения. Результат вычисления уравнений формирует вектор ошибки (синдром). На основе значения синдрома находится позиция потенциальной ошибки в принятом коде.

10. Как проводится декодирование систематического кода по методу полного корректирующего вектора?

Вычисляется корректирующий вектор $C = (c_1, c_2 \dots c_{nk})$, составляющие которого есть скалярные произведения принятого вектора V_x на строки проверочной матрицы H ($c_j = U_j * V_x$).

1. Заранее вычисляются корректирующие вектора для всех возможных векторов ошибки.
2. Для принятого сообщения вычисляется его корректирующий вектор.
3. По таблице соответствий находится вектор ошибки c_j , который привел к такому корректирующему вектору.
4. Ошибка исправляется путём сложения V_x и вектора ошибки.

ВЫВОДЫ

В ходе выполнения лабораторной работы были закреплены теоретические знания по методам кодирования информации. Были получены практические навыки по кодированию и декодированию информационных последовательностей с помощью линейных групповых кодов (ЛГК).

Исходя из полученных результатов, можно сделать следующие выводы:

Определение параметров кода: Для кодирования 256 возможных сообщений требуется информационный блок $n_u = 8$ бит. Для обеспечения возможности обнаружения и исправления одиночной ошибки требуется добавить $n_k = 4$ контрольных разряда для составления ЛГК. Общая длина кода составляет 12 бит.

Построение порождающей матрицы: Кодирование осуществлялось путём составления порождающей матрицы G из единичной матрицы I и проверочной матрицы P .

Исправление одиночной ошибки: В ходе декодирования ЛГК составляется вектор синдрома, указывающий на позицию ошибки в коде, что позволяет корректно обнаружить её наличие и исправить повреждённый бит.